

9. Gradient boosted trees

2.12.2025

ETH zürich

D MATH

In today's class...

9.1 A brief history of boosting and motivation

9.2 Adaptive Boosting (AdaBoost)

9.3 Stage-wise Adaptive Modelling

9.4 Gradient Boosted Trees

9.5 Regularization techniques

9.1 A brief history of boosting

- Early works on boosting [Valiant, 1984, Kearns and Valiant, 1994].
 - *Hypothesis boosting problem*: is there an efficient algorithm to turn weak learners into strong learners?
- AdaBoost [Schapire, 1990, Freund, 1995, Freund and Schapire, 1997]
 - First (and still popular) boosting algorithm for classification.
- Gradient Boosting
 - Boosting as a loss minimization problem [Breiman, 1997]
 - Boosting as functional gradient descent in function space [Mason et al., 1999]
 - Gradient Boosting Machines [Friedman, 2001, Friedman, 2002]
- ... many improved algorithms, most used are: XGBoost [Chen and Guestrin, 2016] and LightGBM [Ke et al., 2017].

9.1 Motivation

Setting:

- Suppose you want to solve a binary classification problem
 $D = \{(x_i, y_i) \in \mathcal{X} \times \{-1, +1\}, 1 \leq i \leq m\}$.
- But you only know how to train weak classifiers (i.e. classifiers that are only slightly better than random guessing).
- Is there a way to combine many weak classifiers into a strong one?

Main idea of boosting:

- Train the weak classifiers sequentially on reweighted data, where larger weights are given to previously misclassified datapoints.
- Aggregate all weak classifiers into a strong classifier (a “committee” of models) by weighing each classifier’s outputs according to its error rate.

9.2 Adaptive Boosting (AdaBoost)

Algorithm: AdaBoost

Input: Training set $\{(x_i, y_i) \in \mathcal{X} \times \{-1, +1\}, 1 \leq i \leq m\}$; number of boosting steps K .

Output: Final classifier C .

Initialize *data weights* $w_i^{(1)} = 1$, for $i = 1, \dots, m$.

for $k = 1, \dots, K$ **do**

- Fit a classifier $C_k = \arg \min_C \sum_{i=1}^m w_i^{(k)} \mathbb{1}_{\{C(x_i) \neq y_i\}}$.
- Compute weighted *error rate* $\text{err}_k := \frac{\sum_{i=1}^m w_i^{(k)} \mathbb{1}_{\{C_k(x_i) \neq y_i\}}}{\sum_{i=1}^m w_i^{(k)}}$
- Compute *classifier's weight* $\alpha_k := \log((1 - \text{err}_k)/\text{err}_k)$
- Update *data weights* $w_i^{(k+1)} := w_i^{(k)} \exp(\alpha_k \mathbb{1}_{\{C_k(x_i) \neq y_i\}})$, for $i = 1, \dots, m$.

end

return $C(x) = \text{sign}\left(\sum_{k=1}^K \alpha_k C_k(x)\right)$.

9.2 Adaptive Boosting (AdaBoost)

- Without loss of generality, we can assume that $\text{err}_k < 1/2$, because if $\text{err}_k \geq 1/2$ then we can “flip” the model’s predictions ($+1 \mapsto -1$ and $-1 \mapsto +1$) and obtain a model with $\text{err}_k < 1/2$.
- Therefore, the *data weights* are increased for misclassified samples:

$$w_i^{(k+1)} := w_i^{(k)} \exp(\alpha_k \mathbb{1}_{\{C_k(x_i) \neq y_i\}}) = \begin{cases} w_i^{(k)} & \text{if } C_k(x_i) = y_i \\ w_i^{(k)} \left(\frac{1 - \text{err}_k}{\text{err}_k} \right) & \text{if } C_k(x_i) \neq y_i \end{cases}$$

$\implies$ the next weak classifier is forced to give “more attention” to misclassified samples.

- The final model $C(x) = \text{sign} \left(\sum_{k=1}^K \alpha_k C_k(x) \right)$ is a committee of all trained weak learners, weighted by their error rates α_k .

9.2 Adaptive Boosting (AdaBoost)

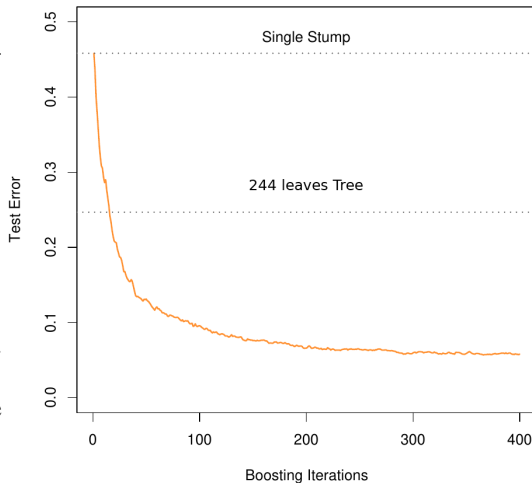
Consider the following classification problem [Hastie et al., 2009]:

- $(X_1, \dots, X_d) \sim \mathcal{N}(0, I_d)$, with $d = 10$

- $Y = \begin{cases} 1 & \text{if } \sum_{j=1}^{10} X_j^2 > \chi, \\ -1 & \text{otherwise,} \end{cases}$

where χ is the median of a chi-squared random variable with 10 degrees of freedom.

We train AdaBoost with tree stumps (i.e. trees with two leaves) as weak classifiers. AdaBoost outperforms a fully trained large classification tree with 244 leaves.



9.3 Stage-wise Adaptive Modelling

- Boosting is equivalent to fitting an *additive expansion* in a set of elementary *basis functions*, i.e. functions of the form:

$$f(x) = \sum_{k=1}^K \beta_k g_{\theta_k}(x),$$

where:

- $g_{\theta}(x)$ is a (usually very simple) parametrized function with parameters θ ,
- $\beta_k \in \mathbb{R}$ are the expansion coefficients.
- In the case of AdaBoost, the elementary basis functions are the weak classifiers $C_k(x)$.
- In this course, we have already seen many examples of additive expansions:
 - Polynomials: $f(x) = \sum_{k=0}^K b_k x^k$.
 - Kernel expansions: $f(x) = \sum_{i=1}^m \alpha_i k(x_i, x)$.
 - (Biasless) shallow neural networks: $f(x) = \sum_{i=1}^n W_{i,1}^{(2)} \rho \left(\sum_{j=1}^d W_{i,j}^{(1)} x_j + b_i^{(1)} \right)$

9.3 Stage-wise Adaptive Modelling

- It is possible to fit an additive expansion by solving

$$\arg \min_{\{\beta_k, \theta_k\}_{k=1}^K} \frac{1}{m} \sum_{i=1}^m \ell \left(\sum_{k=1}^K \beta_k g_{\theta_k}(x_i), y_i \right).$$

- This is typically done with polynomial regression, kernel methods and neural networks.
- But for some models (such as classification and regression trees), there is no obvious way to do this or it might be computationally intensive.
- Instead, we can fit basis functions greedily by using a “forward stage-wise procedure”.

9.3 Stage-wise Adaptive Modelling

Algorithm: Forward Stage-wise Adaptive Modelling

Input: Training set $\{(x_i, y_i) \in \mathcal{X} \times \mathcal{Y}, 1 \leq i \leq m\}$; number of boosting steps K .

Output: Final model f_K .

Initialize model $f_0(x) = 0$.

for $k = 1, \dots, K$ **do**

 Compute:

$$(\beta_k, \theta_k) = \arg \min_{\beta, \theta} \frac{1}{m} \sum_{i=1}^m \ell(f_{k-1}(x_i) + \beta g_{\theta}(x_i), y_i).$$

 Set $f_k(x) = f_{k-1}(x) + \beta_k g_{\theta_k}(x)$.

end

return f_K .

9.3 Stage-wise Adaptive Modelling

- For the square loss $\ell(\hat{y}, y) = (y - \hat{y})^2$, we have:

$$\begin{aligned}\ell(f_{k-1}(x_i) + \beta g_\theta(x_i), y_i) &= (y_i - f_{k-1}(x_i) - \beta g_\theta(x_i))^2 \\ &= (r_{k,i} - \beta g_\theta(x_i))^2\end{aligned}$$

where $r_{k,i} = y_i - f_{k-1}(x_i)$ is the **residual** of the current model on the i -th datapoint.

- Therefore, for the square loss, the k -th expansion term $\beta_k g_{\theta_k}$ is just the least-squares predictor of the residuals.

9.3 Stage-wise Adaptive Modelling

Proposition 1 (AdaBoost is stage-wise adaptive)

AdaBoost is equivalent to Forward Stage-wise Adaptive modelling with:

- *loss function given by $\ell(f(x), y) = \exp(-yf(x))$ (exponential loss function),*
- *basis functions given by the classifiers $C_k(x)$.*

Proof.

At the blackboard. □

- The AdaBoost algorithm was invented before this connection with Forward Stage-wise Adaptive modelling was discovered.
- The simple re-weighting scheme of AdaBoost is a direct consequence of using the exponential loss as loss function. In general other losses do not admit such simple schemes.
- One can show (see Problem Set 6, Exercise 6) that:

$$f^*(x) = \arg \min_f \mathbb{E}[\exp(-Y f(X))] = \frac{1}{2} \log \left(\frac{\mathbb{P}(Y = 1|X)}{\mathbb{P}(Y = -1|X)} \right),$$

i.e. the best predictor for the exponential loss is given by one-half the log-odds of $\mathbb{P}(Y = 1|X)$.

9.4 Gradient Boosted Trees

- We focus on forward stage-wise adaptive modelling using trees as weak learners.
- We want to extend the method to problems with any differentiable loss function (not just AdaBoost's exponential loss) and we want to solve also regression problems.
- We look for models of the form:

$$f_K(x) = \sum_{k=1}^K g_k(x)$$

where $g_k = (T_k, P_k, V_k)$ is a tree of small size.

- This is an additive expansion of trees, where the coefficients β_k are learned directly by each tree $g_k = (T_k, P_k, V_k)$.
- At each step of the forward stage-wise procedure we must solve:

$$g_k = \arg \min_{g=(T,P,V)} \frac{1}{m} \sum_{i=1}^m \ell(f_{k-1}(x_i) + g(x_i), y_i).$$

- Approximate algorithms for this stage-wise optimization can be derived by analogy to gradient descent.

9.4 Gradient Boosted Trees

Algorithm: Gradient Tree Boosting Algorithm

Input: Training set $\{(x_i, y_i) \in \mathcal{X} \times \mathcal{Y}, 1 \leq i \leq m\}$; number of boosting steps K .

Output: Final model f_K .

Initialize weights $f_0(x) = 0$.

for $k = 1, \dots, K$ **do**

 Compute the *pseudo-residuals*:

$$r_{k,i} = - \left[\frac{\partial \ell(\hat{y}, y_i)}{\partial \hat{y}} \right]_{\hat{y}=f_{k-1}(x_i)}$$

 Fit a square loss regression tree $g_k = (T_k, P_k, V_k)$ to $(r_{k,i}, i = 1, \dots, m)$.

 Set $f_k(x) = f_{k-1}(x) + g_k(x)$.

end

return f_K .

9.4 Gradient Boosted Trees

- For square loss, the negative gradient is equal to the residuals $r_{k,i} = y_i - f_{k-1}(x_i)$ (up to a factor 2), which is why it is common to denote the negative gradients as “pseudo-residuals”.
- In general, the weak learners in gradient boosting can belong to any model class (not just trees), but gradient boosted trees have been particularly successful.
- Gradient boosting can be thought of as functional gradient descent: we minimize the loss in the direction of the negative gradient while remaining in the space of additive expansions in the set of weak learners.
- In most implementations, a learning rate for the gradient descent is introduced using the shrinkage parameter (see Section 9.5 “Regularization techniques” below).
- More sophisticated implementations, such as XGBoost and LightGBM, use second-order approximations (e.g. the Newton-Raphson method) for the gradient step.

9.5 Regularization techniques

- Fitting an optimal tree at each boosting step leads to large trees and overfitting. It is therefore very important to control the complexity of the model at each step.

The most common way to control overfitting is to limit the *maximum number of leaves*:

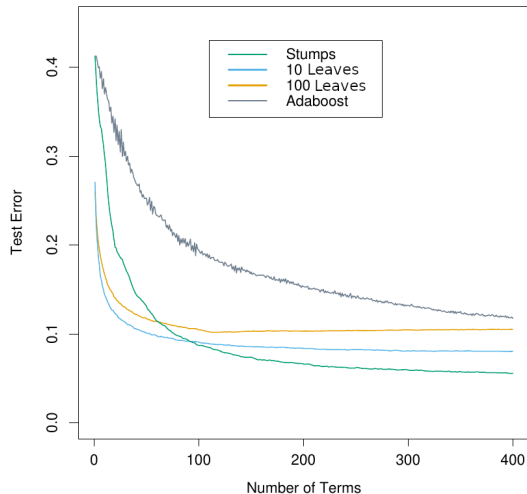
- At boosting step k , we don't fit a full tree at each step and then prune it, rather we restrict its capacity by fixing the number of leaves $J_k := |\mathcal{T}_k|$.
- It is common to choose a unique number of leaves for all steps, i.e. $J_k = J$, for all $k = 1, \dots, K$.
- J quantifies the amount of interaction between features, since no interactions of order greater than $J - 1$ are allowed.
- More specifically, if $\mathcal{X} = \mathbb{R}^d$, then we can expand our model output as:

$$f(x_1, \dots, x_d) = \sum_{j=1}^d f_j(x_j) + \sum_{j,k=1}^d f_{j,k}(x_j, x_k) + \sum_{j,k,l=1}^d f_{j,k,l}(x_j, x_k, x_l) + \dots,$$

where the functions f_j are first-order interactions, $f_{j,k}$ are second-order interactions, etc.

- Real data tend to have mostly low-order interactions, so in practice $4 \leq J \leq 8$ works well.

9.5 Regularization techniques



- Setting: same classification problem we saw for AdaBoost.
- Using trees with too much capacity can cause higher variance, hence overfitting.

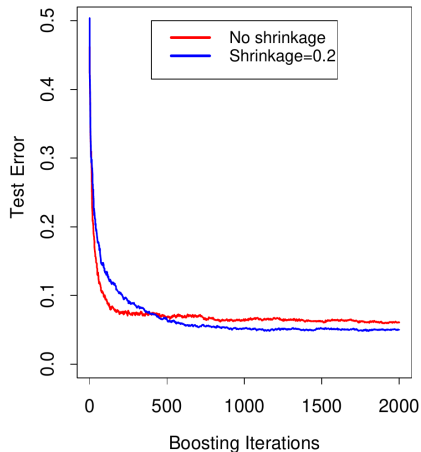
9.5 Regularization techniques

- The *number of boosting steps* K is also an important parameter.
- Gradient boosted trees are normally implemented with early stopping on the parameter K .
 - For fixed $p \in \mathbb{N}$ (the patience parameter), stop training if after p boosting steps the error on a validation set has not decreased.
- Controlling J and K is not the only way to regularize boosted trees. Another commonly used regularization method is *shrinkage*.
- With shrinkage the contribution of each tree is scaled by a factor $\eta \in (0, 1)$.

$$f_k(x) = f_{k-1}(x) + \eta g_k(x).$$

- Shrinkage controls the learning rate of the gradient boosting procedure and can be used to enforce slow learning and avoid overfitting.

9.5 Regularization techniques

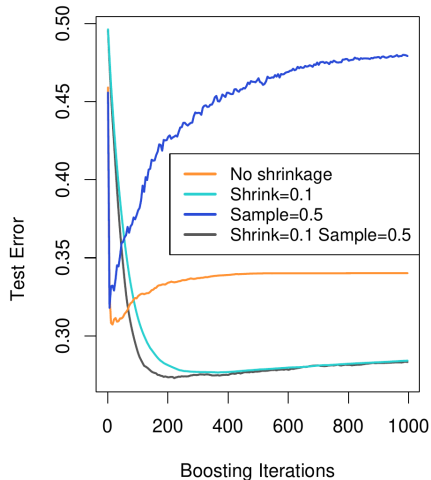


- Empirically shrinkage improves test-error [Friedman, 2001].
- There is a trade-off between shrinkage and early-stopping: smaller values of η result in larger optimal values for K .

9.5 Regularization techniques

- Another regularization technique is *subsampling*, which is a technique adopted from bagging.
- Gradient boosting with subsampling is also known as *stochastic gradient boosting* [Friedman, 2002].
- At each step we train the tree only on a fraction α of the training observations (subsampling without replacement).
- Typically $\alpha = 1/2$, but for large sample sizes it is common to use smaller values.

9.5 Regularization techniques



- Subsampling is useful to reduce computing time, but it also increases performance.
- Subsampling is best used in combination with shrinkage.

References I

-  Breiman, L. (1997).
Arcing the edge.
Technical report, Citeseer.
-  Chen, T. and Guestrin, C. (2016).
Xgboost: A scalable tree boosting system.
In Proceedings of the 22nd acm sigkdd international conference on knowledge discovery and data mining, pages 785–794.
-  Freund, Y. (1995).
Boosting a weak learning algorithm by majority.
Information and computation, 121(2):256–285.
-  Freund, Y. and Schapire, R. E. (1997).
A decision-theoretic generalization of on-line learning and an application to boosting.
Journal of computer and system sciences, 55(1):119–139.

References II

-  Friedman, J. H. (2001).
Greedy function approximation: a gradient boosting machine.
Annals of statistics, pages 1189–1232.
-  Friedman, J. H. (2002).
Stochastic gradient boosting.
Computational statistics & data analysis, 38(4):367–378.
-  Hastie, T., Tibshirani, R., Friedman, J. H., and Friedman, J. H. (2009).
The elements of statistical learning: data mining, inference, and prediction, volume 2.
Springer.
-  Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q., and Liu, T.-Y. (2017).
Lightgbm: A highly efficient gradient boosting decision tree.
Advances in neural information processing systems, 30.
-  Kearns, M. and Valiant, L. (1994).
Cryptographic limitations on learning boolean formulae and finite automata.
Journal of the ACM (JACM), 41(1):67–95.

References III

-  Mason, L., Baxter, J., Bartlett, P., and Frean, M. (1999).
Boosting algorithms as gradient descent.
Advances in neural information processing systems, 12.
-  Schapire, R. E. (1990).
The strength of weak learnability.
Machine learning, 5:197–227.
-  Valiant, L. G. (1984).
A theory of the learnable.
Communications of the ACM, 27(11):1134–1142.